Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en sistemas de información y ciencias de la computación

Desarrollo web

Sección “C”

Implementación del Proyecto de Seguridad con Spring Security y JWT

NOMBRE Y APELLIDO	CARNÉ
Angie Dayana Jacinto Soyos	9989-23-9752

Contenido

1. Descripción de la arquitectura implementada. 3
   - Servicio de autenticación: auth-service. 3
   - API de negocio: apirest-demo 3
   - Comunicación y seguridad entre servicios 4
2. Diagrama o explicación de cómo interactúan el proyecto de seguridad y el backend de clínicas. 4
3. Capturas de la estructura del proyecto. 5
4. Capturas de las entidades de seguridad (Usuarios, Roles y demás entidades implementadas). 6
5. Evidencia de la generación de las llaves RSA. 6
6. Capturas de las pruebas realizadas utilizando Swagger. 7
7. Evidencia de que el backend de clínicas valida correctamente el token mediante la llave pública. 9
   - Iniciar Sesión y obtener token 9
   - Sin token 9
   - Con token 10
8. Una breve explicación del flujo de autenticación y autorización implementado. 11
   - Registro de un usuario 11
   - Inicio de sesión y generación del JWT 11
   - Acceso a los recursos protegidos 12
   - Autenticación y autorización efectiva 12

1. Descripción de la arquitectura implementada.

La solución está compuesta por dos aplicaciones Spring Boot independientes que trabajan de manera coordinada.

El servicio auth-service se encarga de registrar usuarios, comprobar credenciales y emitir tokens JWT. Por su parte, apirest-demo expone los recursos de pacientes y enfermedades y actúa como servidor de recursos, por lo que valida el token antes de permitir el acceso a sus endpoints. Esta separación distribuye las responsabilidades de identidad y de negocio entre servicios diferentes.

Ambas aplicaciones siguen una arquitectura en capas. Las solicitudes HTTP llegan a los controladores REST; estos delegan la lógica a los servicios; los servicios utilizan repositorios Spring Data JPA para acceder a la base de datos H2. La comunicación es stateless: no se mantiene una sesión de usuario en el servidor y cada petición protegida debe presentar su propio token.

Aplicación	Puerto	Responsabilidad principal	Tecnologías relevantes
auth-service	8081	Registro, inicio de sesión, gestión de usuarios/roles/permisos y emisión de JWT.	Spring Boot 3.2.5, Spring Security, JPA, H2, BCrypt, Nimbus JOSE/JWT y RSA.
apirest-demo	8082	Operaciones REST sobre pacientes y enfermedades y validación de JWT.	Spring Boot 4.1.0, OAuth2 Resource Server, JPA, H2, MapStruct y Swagger/OpenAPI.

o Servicio de autenticación: auth-service

auth-service centraliza la identidad de los usuarios. AuthController publica los endpoints /api/v1/auth/login y /api/v1/auth/register. AuthService contiene la lógica de validación de credenciales, registro y asignación de roles, mientras que TokenService construye y firma los JWT. UsuarioRepository, RolRepository y PermisoRepository permiten persistir el modelo de seguridad mediante JPA.

- Usuario: almacena username, contraseña cifrada, correo, estado habilitado y roles.
- Rol: agrupa permisos y se relaciona con los usuarios mediante una asociación muchos a muchos.
- Permiso: representa capacidades como READ_PRIVILEGE, WRITE_PRIVILEGE y DELETE_PRIVILEGE.
- JwtConfig: carga las claves RSA y configura el codificador JWT y BCrypt.
- DataSeeder: crea roles, permisos y usuarios iniciales para el entorno de demostración.

o API de negocio: apirest-demo

apirest-demo implementa la lógica de pacientes y enfermedades. Cada dominio cuenta con controlador, servicio, repositorio, entidad, DTO y mapper. Los controladores exponen operaciones para crear, listar, consultar por identificador y eliminar registros. Los mappers de MapStruct separan los objetos utilizados por la API de las entidades JPA, y los servicios delimitan las operaciones transaccionales antes de acceder a H2.

Capa	Componentes	Función
Seguridad	SecurityConfig y NimbusJwtDecoder	Interceptar solicitudes y validar la firma y vigencia del JWT.
Controladores	PacienteController y EnfermedadController	Recibir solicitudes HTTP y devolver respuestas REST.

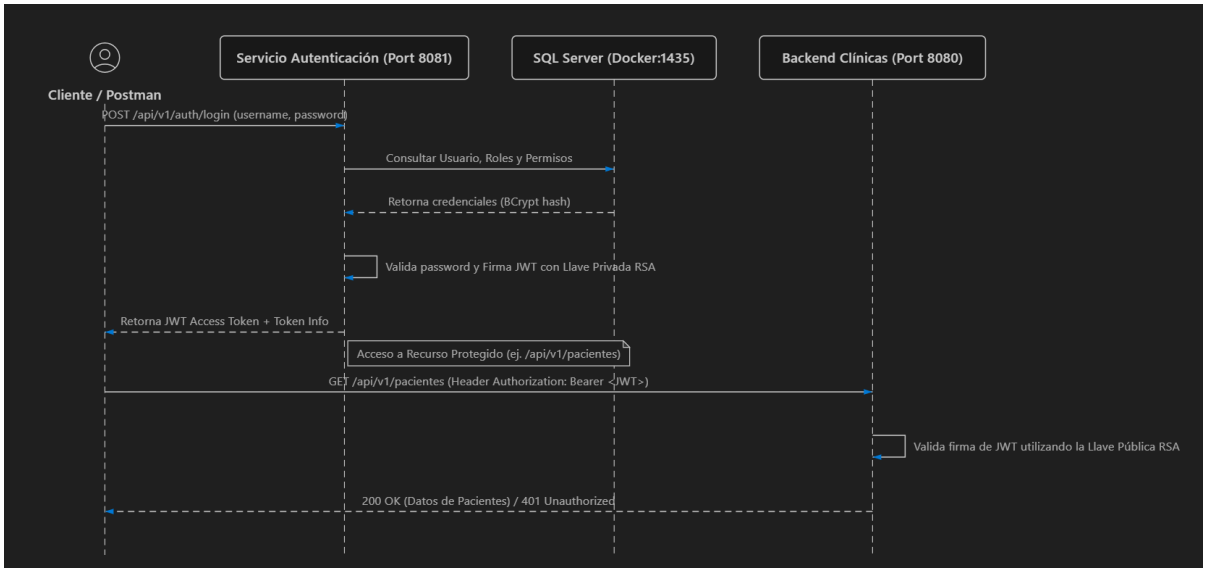
Capa	Componentes	Función
Mapeo	PacienteMapper y EnfermedadMapper	Convertir DTOs en entidades y entidades en DTOs.
Servicios	PacienteServiceImpl y EnfermedadServiceImpl	Aplicar la lógica de aplicación y controlar transacciones.
Persistencia	Repositories, JPA/Hibernate y H2	Guardar, consultar y eliminar información.

- ### Comunicación y seguridad entre servicios

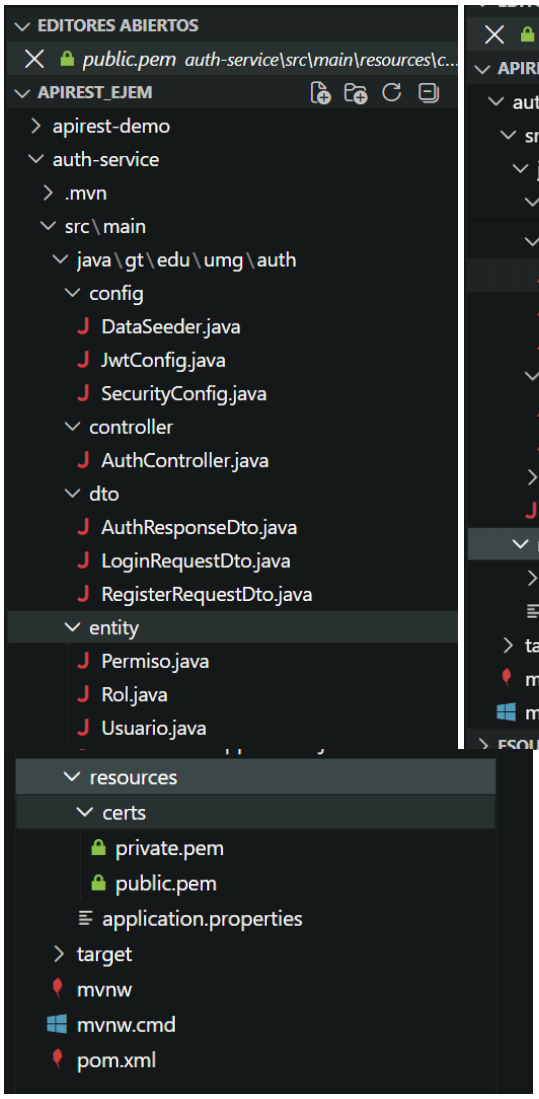
Los servicios comparten un esquema de criptografía asimétrica. auth-service conserva la clave privada y la utiliza para firmar cada JWT; apirest-demo dispone de la clave pública correspondiente y la utiliza para verificar que el token fue emitido por el servicio de autenticación y que no fue alterado. La clave pública permite validar firmas, pero no permite generar tokens válidos.

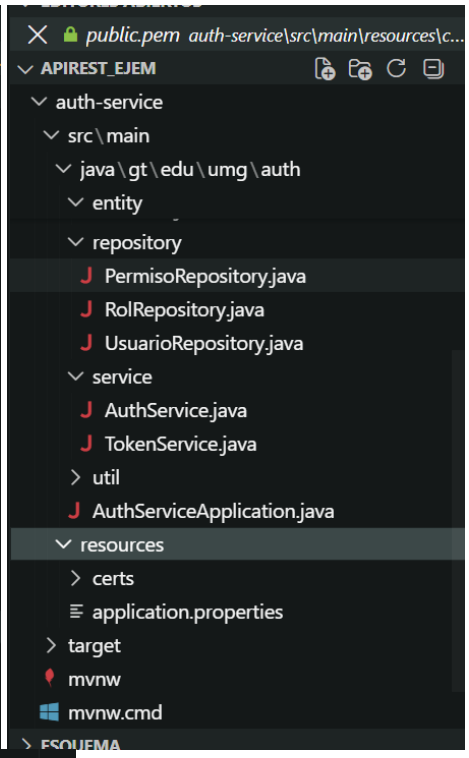
El filtro de Spring Security de apirest-demo permite el acceso público a Swagger y a la consola H2. Para el resto de las rutas exige que la solicitud esté autenticada. Después de superar esta validación, la petición continúa hacia el controlador, el servicio y el repositorio correspondiente.

2. Diagrama o explicación de cómo interactúan el proyecto de seguridad y el backend de clínicas.

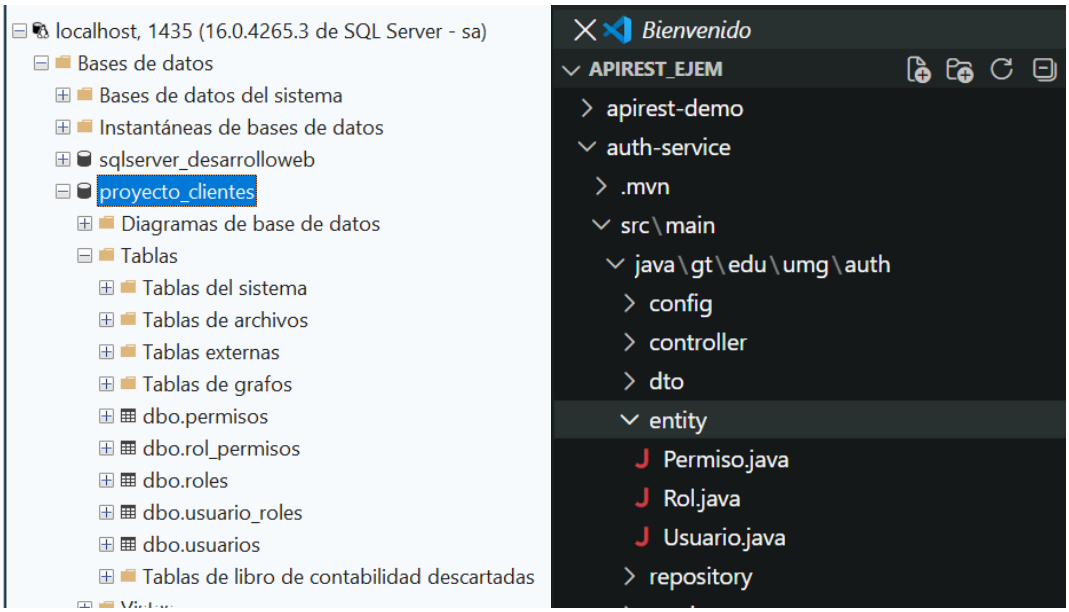


3. Capturas de la estructura del proyecto.

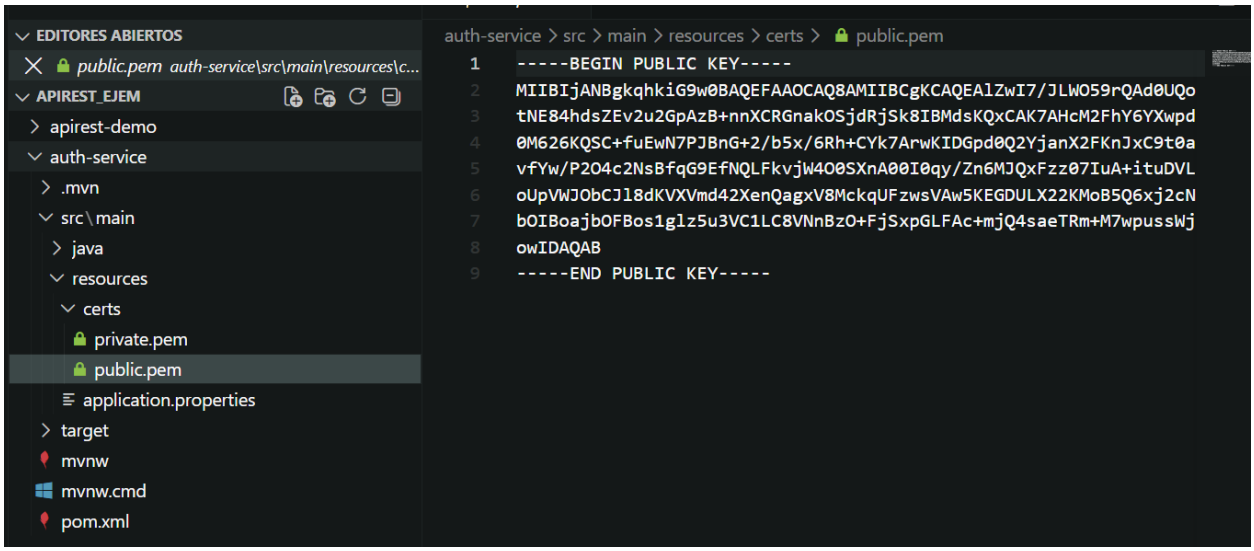




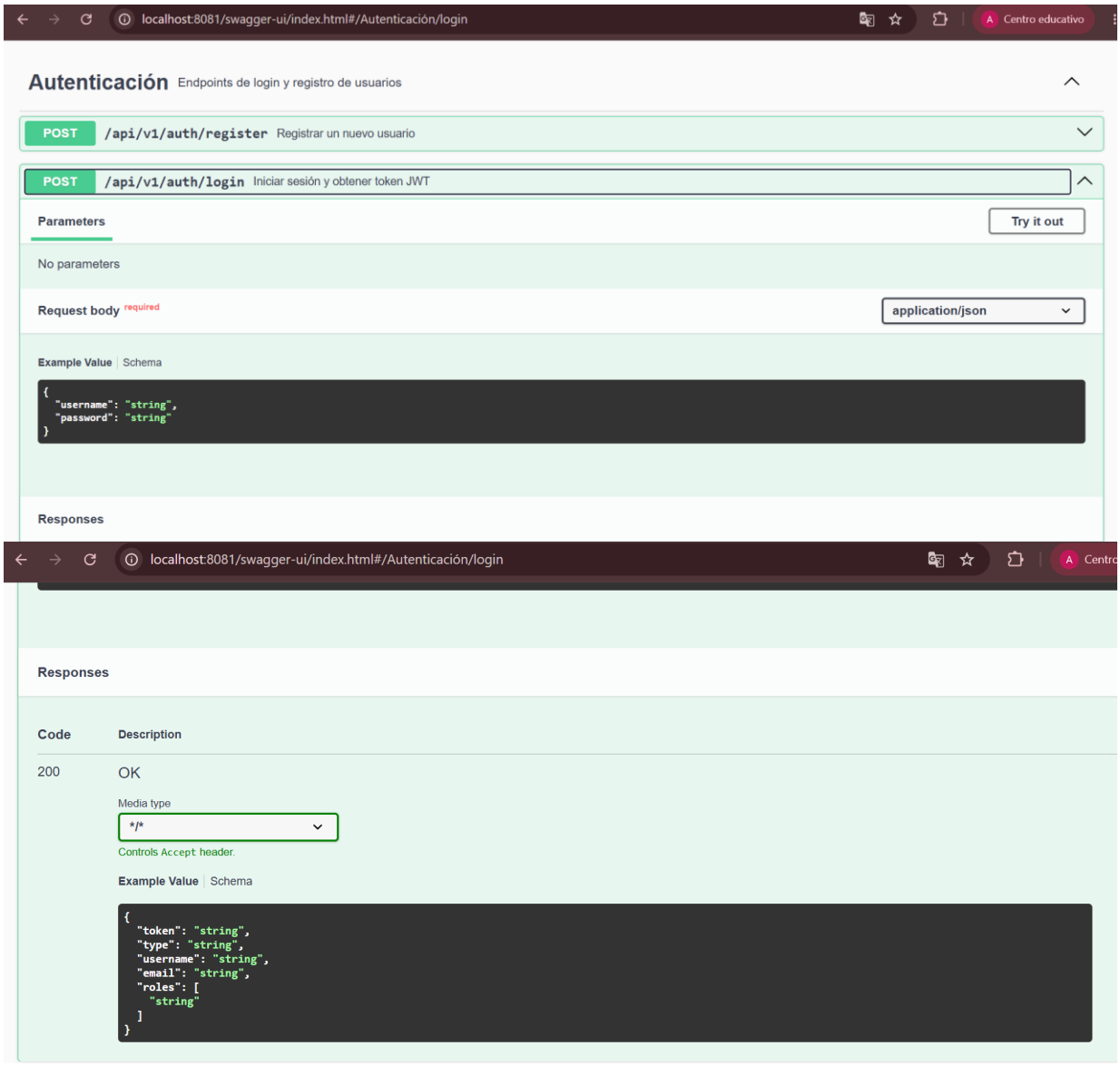
4. Capturas de las entidades de seguridad (Usuarios, Roles y demás entidades implementadas).

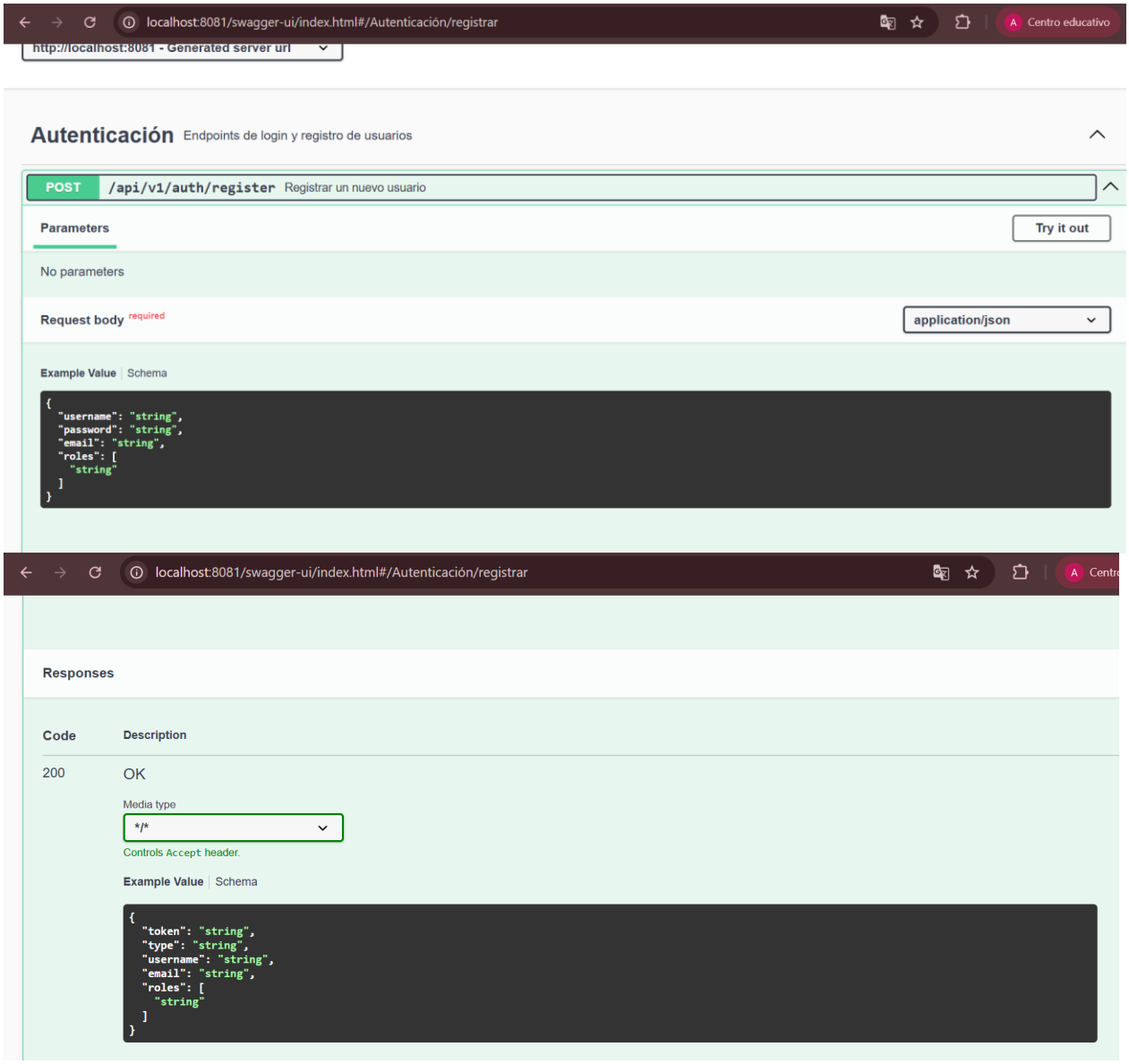


5. Evidencia de la generación de las llaves RSA.



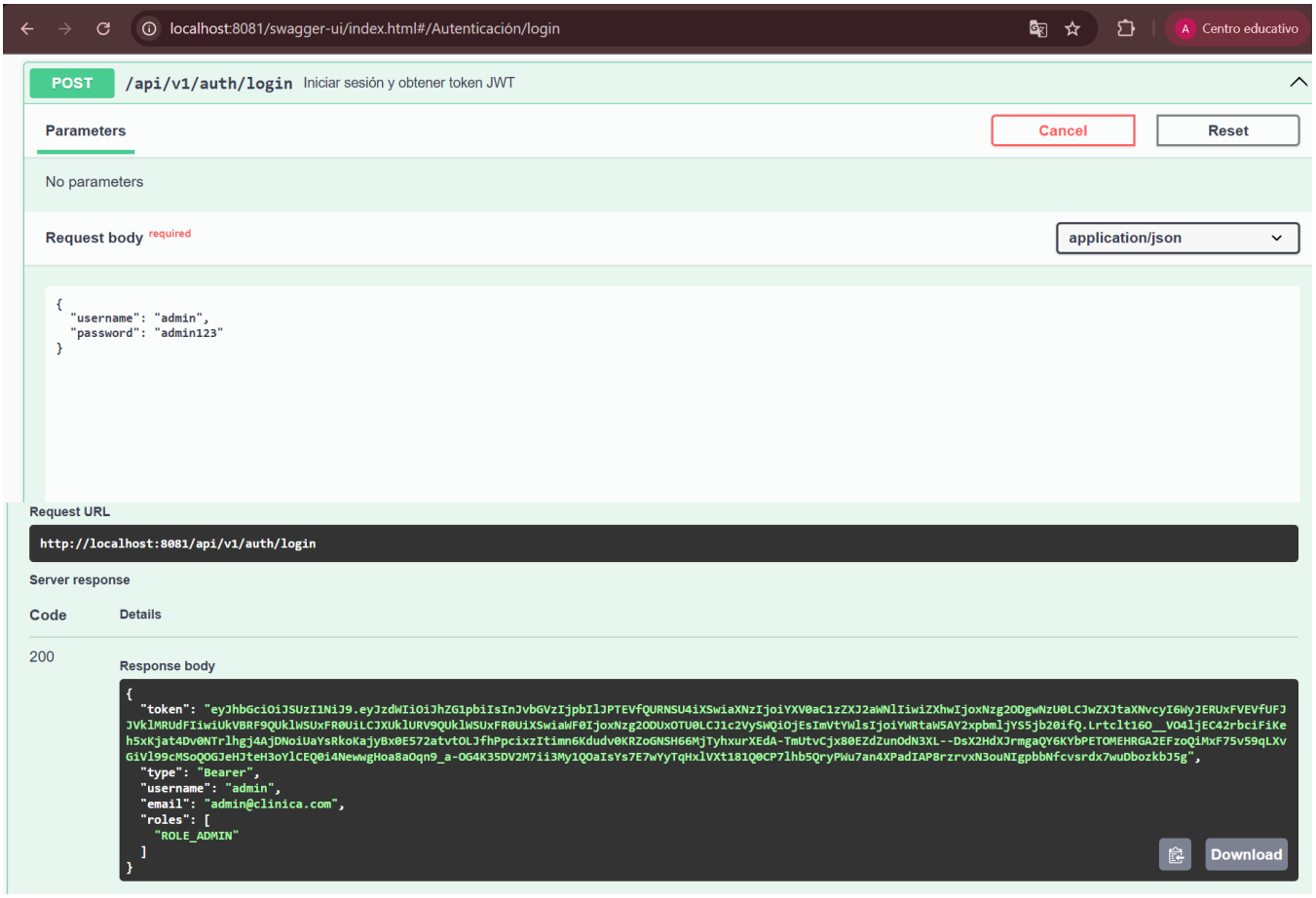
6. Capturas de las pruebas realizadas utilizando Swagger.



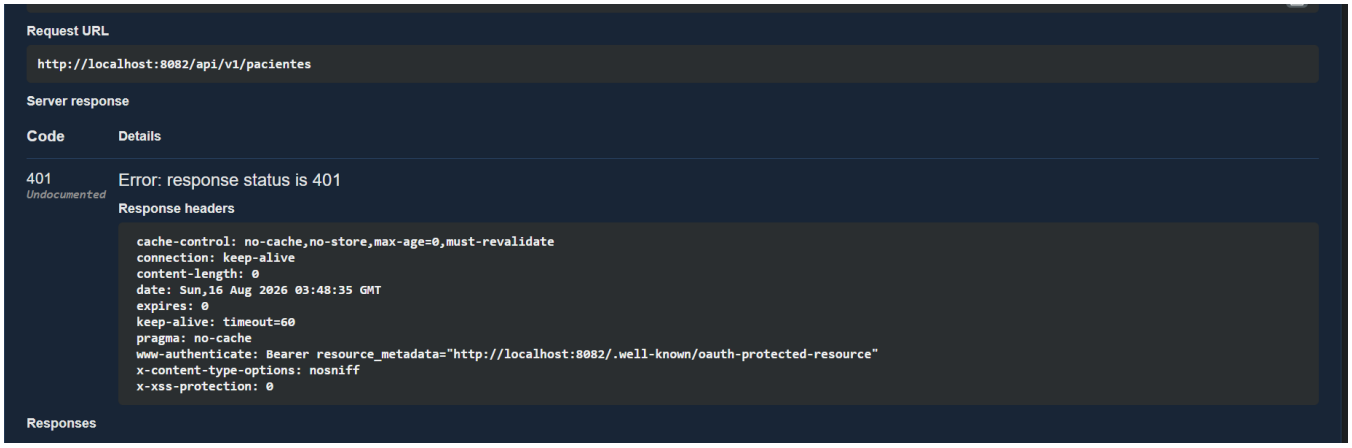

localhost:8081/swagger-ui/index.html#/Autenticación/registrar
Centro educativo
http://localhost:8081 - Generated server url
Autenticación
Endpoints de login y registro de usuarios
POST
/api/v1/auth/register
Registrar un nuevo usuario
Parameters
Try it out
No parameters
Request body
required
application/json
Example Value
Schema
{
"username": "string",
"password": "string",
"email": "string",
"roles": [
"string"
]
}
localhost:8081/swagger-ui/index.html#/Autenticación/registrar
Responses
Code
Description
200
OK
Media type
/
Controls Accept header.
Example Value
Schema
{
"token": "string",
"type": "string",
"username": "string",
"email": "string",
"roles": [
"string"
]
}

7. Evidencia de que el backend de clínicas valida correctamente el token mediante la llave pública.

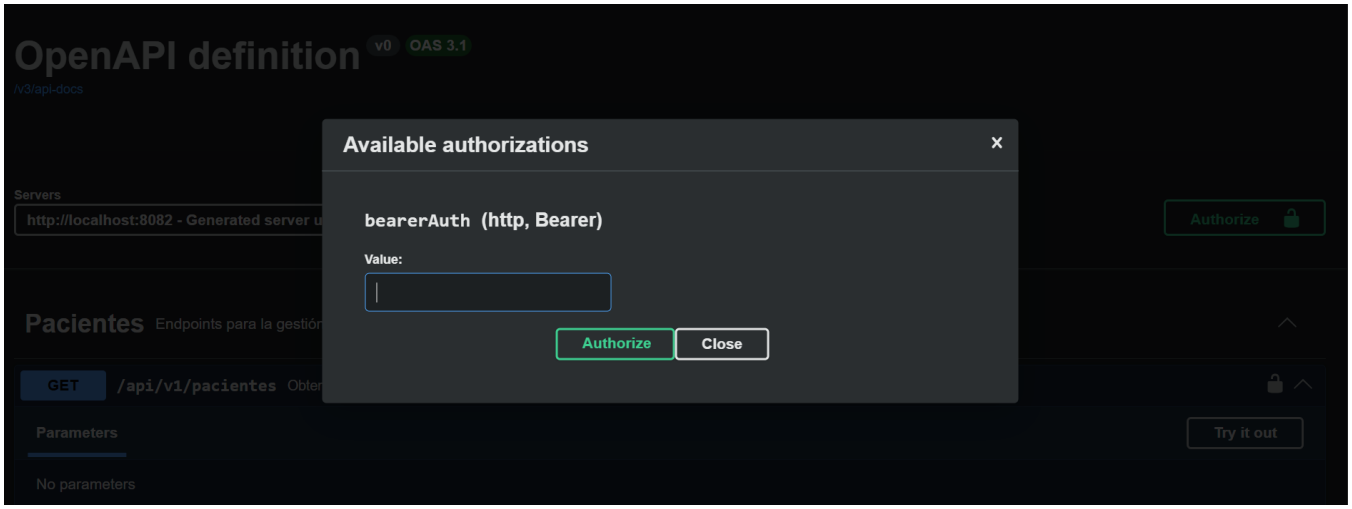
- ### Iniciar Sesión y obtener token

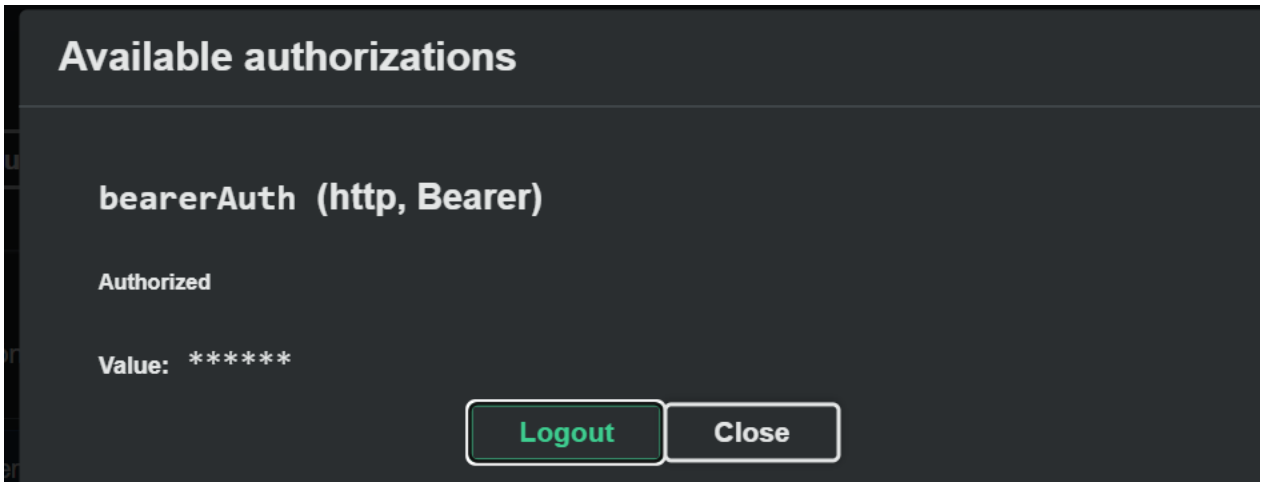


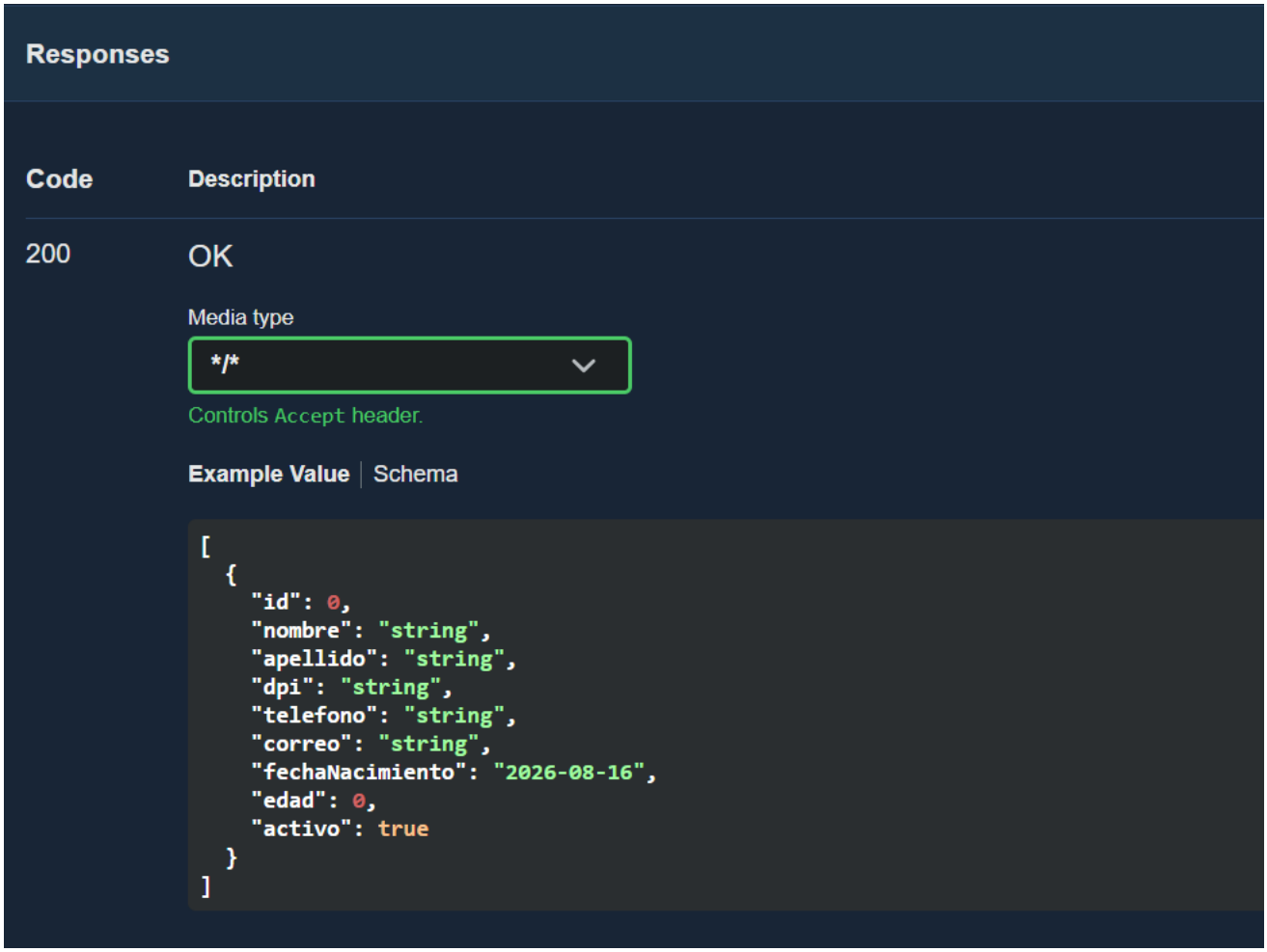
- ### Sin token



- **Con token**


OpenAPI definition
v0
OAS 3.1
/v3/api-docs
Servers
http://localhost:8082 - Generated server u
Authorize
Available authorizations
bearerAuth (http, Bearer)
Value:
Authorize
Close
Pacientes
Endpoints para la gestión
GET
/api/v1/pacientes
Parameters
Try it out
No parameters


Available authorizations
bearerAuth (http, Bearer)
Authorized
Value: ******
Logout
Close



8. Una breve explicación del flujo de autenticación y autorización implementado.

- **Registro de un usuario**
  - Recepción. El cliente envía username, password, email y, opcionalmente, roles a POST /api/v1/auth/register.
  - Validación. AuthController aplica las restricciones del DTO y AuthService comprueba que el usuario y el correo no estén registrados.
  - Protección de la contraseña. La contraseña se transforma mediante BCrypt antes de almacenarse; nunca se guarda en texto plano.
  - Persistencia. El usuario se asocia con sus roles y se guarda en la base de datos H2 mediante UsuarioRepository.
  - Emisión. TokenService genera inmediatamente un JWT y el servicio responde con estado HTTP 201 Created.
- **Inicio de sesión y generación del JWT**
  - Solicitud. El cliente envía username y password a POST /api/v1/auth/login.
  - Búsqueda. AuthService localiza al usuario por su nombre y verifica que la cuenta se encuentre habilitada.
  - Comprobación. BCrypt compara la contraseña recibida con el hash almacenado en la base de datos.

- Construcción. Si las credenciales son válidas, TokenService crea un JWT con una vigencia de ocho horas.
- Respuesta. El cliente recibe el token, el tipo Bearer, el nombre de usuario, el correo y sus roles.

- **Acceso a los recursos protegidos**
  - Envío del token. El cliente agrega el encabezado Authorization: Bearer <token> al solicitar un recurso de apirest-demo.
  - Intercepción. Spring Security procesa la solicitud antes de que llegue al controlador REST.
  - Validación. NimbusJwtDecoder usa public.pem para comprobar la firma RSA y verifica la vigencia temporal del JWT.
  - Autenticación. Si el token es válido, Spring crea el contexto de autenticación de la solicitud y permite que continúe.
  - Procesamiento. El controlador delega al mapper, servicio y repositorio para ejecutar la operación solicitada.
  - Rechazo. Si el token falta, está vencido, fue modificado o su firma no es válida, la API rechaza la solicitud antes de ejecutar la lógica de negocio.

Método	Ruta protegida	Operación
POST	/api/v1/pacientes	Crear paciente
GET	/api/v1/pacientes	Listar pacientes
GET	/api/v1/pacientes/{id}	Consultar paciente
DELETE	/api/v1/pacientes/{id}	Eliminar paciente
POST	/api/v1/enfermedades	Crear enfermedad
GET	/api/v1/enfermedades	Listar enfermedades
GET	/api/v1/enfermedades/{id}	Consultar enfermedad
DELETE	/api/v1/enfermedades/{id}	Eliminar enfermedad

- **Autenticación y autorización efectiva**

La autenticación responde a la pregunta de quién realiza la solicitud. En esta solución, se completa cuando auth-service valida las credenciales y cuando apirest-demo acepta el JWT firmado. La autorización responde a qué operaciones puede ejecutar el usuario una vez identificado.

Aunque el token contiene roles y permisos, la configuración actual de apirest-demo solo aplica la regla anyRequest().authenticated(). No existen reglas hasRole o hasAuthority, ni anotaciones como @PreAuthorize en los controladores. En consecuencia, cualquier usuario que presente un JWT válido puede ejecutar todas las operaciones protegidas de pacientes y enfermedades, independientemente de que posea ROLE_USER o ROLE_ADMIN.